Алгоритм Борувки на SPLA и Apache Spark

Шальнев Владислав, Кадочников Даниил

SPLA — разреженная линейная алгебра

SPLA (репозиторий, C++ API) — MIT, **разреженная линейная алгебра** на **GPU (OpenCL)** и **CPU**.

- **Производительность**: типовые ядра (**m×v**, редукции, маски) на GPU эффективнее ручных циклов по графу
- **Semiring**: одни и те же примитивы под разные задачи — свои `×` / `+` и типы
- **Портируемость**: OpenCL, выбор устройства; API **C/C++** и **Python (pyspla)**

Apache Spark

Apache Spark (spark.apache.org) — **распределённый** движок вычислений поверх кластера (**Scala/Java/Python/R**); ориентир на **память** и **план запросов** вместо лишних обращений к диску.

- **Масштаб и устойчивость**: разбиение данных по узлам, перезапуск задач при сбоях, рост почти «добавили воркеры»
- **Стек в одном месте**: **Spark SQL**, **DataFrame/Dataset**, потоки (**Structured Streaming**), **MLlib** — без сшивания десятка разных систем
- **Графы**: **GraphX** и супер-шаги в духе **Pregel** — итеративные обходы и агрегации по распределённому графу

Алгоритм Борувки

Задача: найти **минимальное остовное дерево** (MST) взвешенного связного графа.

Шаги одной итерации:

1. Для каждой компоненты найти минимальное исходящее ребро
2. Добавить найденные рёбра в MST
3. Объединить компоненты, соединённые выбранными рёбрами

Повторяем, пока не останется одна.

Борувка в терминах линейной алгебры

Данные:

- S — матрица смежности: $S[i, j] = w(i, j)$
- $parent[v]$ — корень компоненты вершины v
- Ребро — кортеж $(w,\ i)$: w — вес, i — компонента соседа
- $edge[v]$ — мин. ребро из вершины v в другую компоненту
- $cedge[v]$ — мин. ребро из компоненты v наружу (только для корней, иначе $+\infty$)

Полукольцо $combMin$:

- $\otimes$: $comb\big((w_1, i_1),\ (w_2, i_2)\big) = (w_1,\ i_2)$ — взять вес из первого кортежа, индекс из второго
- $\oplus$: min — выбрать кортеж с меньшим весом

Борувка в терминах линейной алгебры

Инициализация: $parent[v] = v$

while $S \neq \emptyset$:

1. $edge = S \times_{combMin} parent$
   — для каждой вершины найти мин. ребро наружу через mxv
2. $cedge[parent[i]] = min\,(cedge[parent[i]],\ edge[i])$
   — для каждой компоненты выбрать мин. ребро из всех её вершин
3. Обновить $parent$: корни переходят к соседней компоненте, сжатие пути
4. $S = select(S,\ parent[i] \neq parent[j])$
   — удалить рёбра внутри компонент

Борувка в терминах Apache Spark

Данные:

- $G = (V, E)$ — GraphX граф с атрибутом вершины $comp[v]$
- `aggregateMessages` — для рассылки/агрегации рёбер между вершинами
- `collectAsMap` — сбор vertex → component маппинга на драйвер
- `UnionFind` — структура для объединения компонент (на драйвере)

Борувка в терминах Apache Spark

Инициализация: $comp[v] = v$

while есть межкомпонентные рёбра:

1. `aggregateMessages` — каждая вершина отправляет соседям своё минимальное ребро; получатель выбирает минимум
2. `collectAsMap` — собрать карту (вершина → компонента)
3. Для каждой компоненты выбрать минимальное исходящее ребро
4. `UnionFind.union` — объединить компоненты на драйвере
5. `outerJoinVertices` — обновить id компонент у всех вершин
6. `subgraph` — удалить рёбра внутри одной компоненты

Эксперимент 1

Цель: сравнить время работы алгоритма Борувки на SPLA (CPU, NVIDIA OpenCL и Intel OpenCL) и Apache Spark на реальных графах разного размера

Эксперимент 2

Цель: исследовать влияние типа графа на относительную производительность реализаций алгоритма Борувки на SPLA (CPU) и Apache Spark

Вопрос: зависит ли соотношение времени работы SPLA и Spark от структурных свойств графа (дорожная сеть, триангуляция, социальная сеть)?

Ход экспериментов

Эксперимент 1 — влияние размера графа

- 20 запусков алгоритма на каждом графе и библиотеке
- Вычисление среднего времени и доверительных интервалов
- Визуализация и анализ результатов

Эксперимент 2 — влияние типа графа

- 5 запусков алгоритма на каждом графе и библиотеке, 3 прогрева
- Сравнение SPLA (CPU) и Spark
- Визуализация и анализ результатов

Данные Эксперимент 1

Граф	Количество вершин	Количество рёбер
amazon0601	403 394	2 443 408
citationCiteseer	268 495	1 156 647
coPapersDBLP	540 486	15 245 729
in-2004	1 382 908	13 591 473
cit-Patents	3 774 768	16 518 947

Источники: A High-Performance MST Implementation for GPUs 2023, SuiteSparse Matrix Collection

Данные Эксперимент 2

Тип	Граф	Вершины	Рёбра
Road	road-luxembourg-osm	114 599	119 666
Road	road-usroads-48	126 146	161 950
Road	road-roadNet-PA	1 087 562	1 541 514
Geometric	delaunay_n17	131 072	393 176
Geometric	delaunay_n18	262 144	786 396
Geometric	delaunay_n20	1 048 576	3 145 686
Social	soc-LiveMocha	104 103	2 193 083
Social	ca-MathSciNet	332 689	820 644
Social	soc-lastfm	1 191 805	4 519 330

Источник: Network Repository

Характеристики машины Эксперимент 1

ОС: Ubuntu 24.04 LTS

CPU: 10th Gen Intel Core i5-10300H @ 2.50 GHz

- Hyper-Threading включен
- L1: 256 КБ, L2: 1 МБ, L3: 8 МБ

GPU: NVIDIA GeForce GTX 1650 Ti Mobile

- 4 ГБ VRAM GDDR6
- драйвер 580.126.09
- CUDA Toolkit 13.0.97

RAM: 16 ГБ DDR4, 3200 МГц

Характеристики машины Эксперимент 2

ОС: Windows 10, 64-разрядная

CPU: 12th Gen Intel Core i5-1235U @ 1.30 GHz

- 10 ядер (2P + 8E), 12 потоков

GPU: Intel Iris Xe Graphics (128 MB, интегрированная)

RAM: 32 ГБ